

Enterprise Service Intake

Architecture & Design Brief

Candidate	Forrest Zhang
Role	Power Platform Senior Developer - Take-Home Case
Prepared For	Mitacs technical review team
Environment	https://mitacs.crm.dynamics.com/
Portal	https://enterprise-service-intake-hellox.powerappsportals.com
Status	Interview submission artifact

Executive Summary

This solution implements an enterprise-grade external service request intake process using Power Pages, Dataverse, Power Automate, C# plugins, and PCF. External customers submit authenticated multi-step requests, receive real-time routing and SLA feedback, and upload supporting documentation. Dataverse holds the system of record while server-side plugins apply non-bypassable routing and close-validation rules. Power Automate handles manager approval, mock ERP synchronization, and resilient error logging.

End-to-End Outcome

- 1 Customer submits a portal request and receives a formatted confirmation number.
- 2 Dataverse plugin applies the matching routing/SLA rule and flags approval or documentation requirements.
- 3 Internal coordinator reviews the request in the model-driven app with a PCF status indicator.
- 4 Critical or high-priority items route to manager approval.
- 5 Approved requests are posted to a mock REST endpoint and the external system ID is written back to Dataverse.
- 6 Failures in approval or integration are captured in a custom System Error Log table.

Reviewer Access

The environment, portal, and repository are prepared so reviewers can walk through the complete vertical slice.

Area	Value	Reviewer Use
Environment	https://mitacs.crm.dynamics.com/	Open Dataverse tables, model-driven app, flow runs, and solution components.
Maker Solution	https://make.powerapps.com/environments/99dd50ed-a753-e37f-912c-78a022b12b09/solutions	Review solution-aware components and exports.
Model-driven App	https://mitacs.crm.dynamics.com/main.aspx?appid=3de4f813-b454-f111-bec7-000d3a3aca8f	Review request queue, form fields, and PCF/internal user experience.
Power Pages Site	https://enterprise-service-intake-hellox.powerappsportals.com	Submit a customer request and verify dynamic SLA/routing preview.
Cloud Flow	ESI - Approval and ERP Sync	Review approval, HTTP sync, run history, and Try/Catch error handling.

Reviewer Accounts

Account	Purpose
forrest@hellosmart.ca	Admin/customizer reviewer
agent@hellosmart.ca	Internal service coordinator
manager@hellosmart.ca	Approval manager

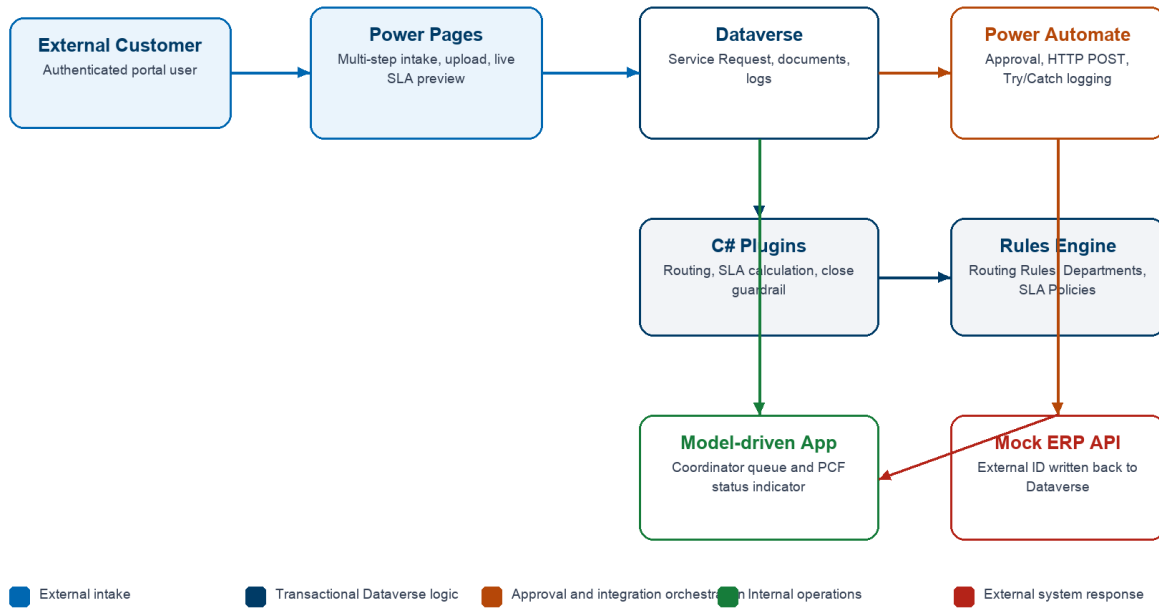
Credential handling: passwords are intentionally not stored in Git or in this document. The system administrator should share them out of band and rotate them after the interview.

Architecture Overview

The solution keeps Dataverse as the authority for state while placing business logic in the component that best enforces the requirement.

Enterprise Service Intake - Runtime Architecture

Power Pages intake, Dataverse system of record, plugin guardrails, approval flow, and mock ERP sync



Requirement	Component	Reason
External intake	Power Pages	Authenticated customer access, contact-scoped permissions, multi-step forms, upload support, and Liquid/Web API extensibility.
System of record	Dataverse	Relational model, security roles, ownership, auditing, solution packaging, and consistency across portal, app, flow, and APIs.
Routing and SLA	C# plugin	Runs transactionally when rows are created or updated and cannot be bypassed through imports, API calls, flows, or alternate clients.
Close guardrail	C# plugin	Enforces critical-request documentation requirements at the server boundary.
Approval and ERP sync	Power Automate	Human approval, HTTP action, connector run history, retry behavior, and clear Try/Catch observability belong in orchestration.
Internal coordinator UX	Model-driven app + PCF	Secure operational CRUD with concise visual severity/SLA/approval status.

Dataverse Data Strategy

The data model separates request state, rule configuration, documents, integration telemetry, and operational error records.

Dataverse Entity Relationship Diagram

Core tables support request intake, dynamic routing, SLA assignment, integration logs, and error handling.

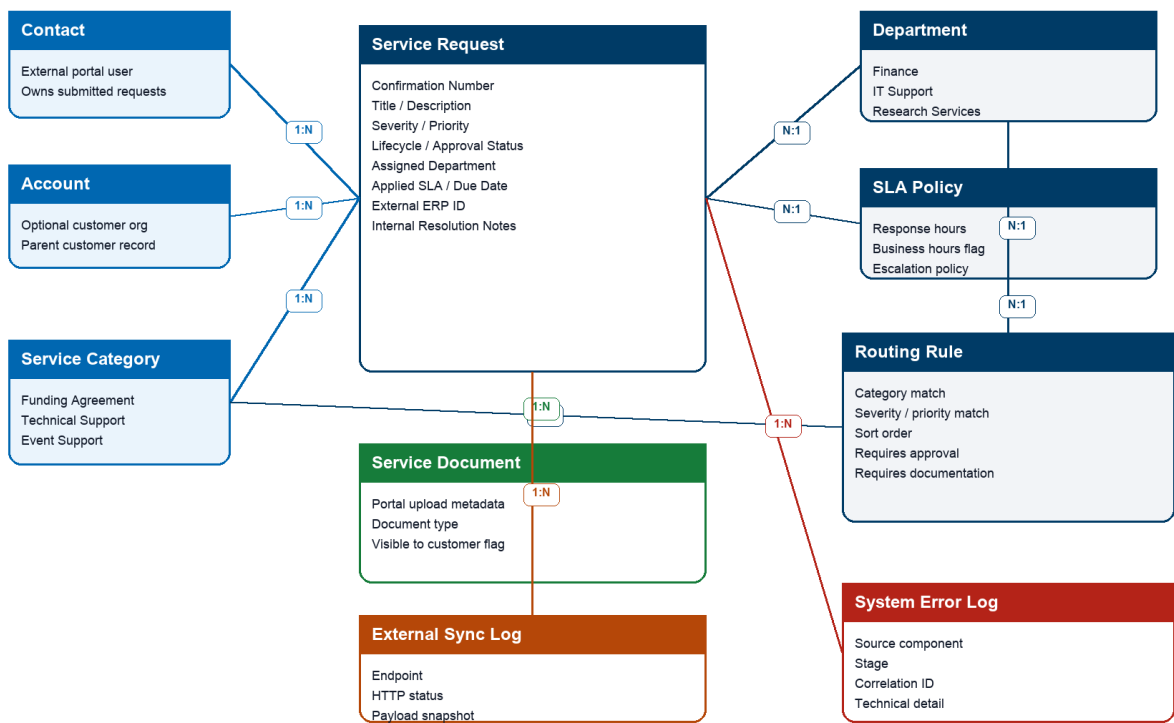


Table	Purpose	Key Relationships
Service Request	Primary intake and work-management row. Stores confirmation number, status, priority, SLA, assigned department, approval state, integration state, external ERP ID, and internal notes.	Contact, Account, Service Category, Department, SLA Policy, Service Document, External Sync Log, System Error Log
Routing Rule	Configurable rules engine row. Matches category/severity/priority and supplies department, SLA, approval requirement, and documentation requirement.	Service Category, Department, SLA Policy
Department	Internal routing destination for operational ownership.	Service Request, Routing Rule
SLA Policy	Response target and escalation policy assigned by routing.	Service Request, Routing Rule
Service Document	Supporting documentation metadata for portal uploads.	Service Request
External Sync Log	Integration audit trail for outbound ERP calls.	Service Request
System Error Log	Structured error capture for plugin, flow, and integration failures.	Service Request optional

Confirmation Number

- Service Request uses a Dataverse autonumber format: SR-{yyyyMMdd}-{SEQNUM}.
- The number is generated server-side, remains stable after creation, and is safe to show externally.

- The formatted value gives customers a readable reference while preserving Dataverse row IDs for integration and support work.

Security Strategy

The design assumes least privilege across external portal users, internal coordinators, managers, and administrators.

Power Pages Access

- External users authenticate to Power Pages and are associated to Contact rows.
- Table permissions are contact-scoped so users can create requests and view only requests tied to their contact.
- Reference data needed for dynamic preview is exposed read-only.
- Portal forms exclude internal-only fields, integration payloads, internal notes, sync logs, and system error logs.

Internal Dataverse Access

- Coordinators use the model-driven app for operational request triage.
- Managers review approvals and can inspect high-priority items.
- Sensitive internal resolution notes are kept off the portal and should be protected with role and column-level security in production.
- Error and sync telemetry tables are intended for managers, admins, and support owners rather than broad staff access.

Security Tradeoffs

Area	Lean Demo Choice	Production Hardening
Portal identity	Authenticated Power Pages users in the interview tenant.	Use Entra External ID or a configured B2C provider, lifecycle controls, and conditional access where appropriate.
Uploads	Portal upload step captures supporting documentation metadata.	Apply file type limits, malware scanning, retention rules, and DLP review.
Secrets	No tenant passwords or tokens committed to Git.	Use connection references, environment variables, Key Vault-backed custom connectors, and managed identities where available.

External User Experience

The portal is scoped as a clean customer intake experience rather than an internal operations surface.

Submission Flow

- 1 User signs in to the Power Pages site.
- 2 User starts a new service request and enters request details.
- 3 User selects category, severity, and priority.
- 4 Portal dynamically previews expected department, SLA target, approval requirement, and documentation requirement before final submission.
- 5 User uploads supporting documentation where needed.
- 6 User reviews and submits the request, then receives a confirmation number.

Dynamic Preview Implementation

- The portal reads eligible routing/SLA data through Power Pages Web API and/or Liquid-rendered data.
- Client-side JavaScript recalculates the displayed routing destination and expected SLA when the user changes category, severity, or priority.
- The preview avoids a full page reload and mirrors the server-side plugin rule result, while the plugin remains the authoritative enforcement layer.

Automation And Integration

The cloud flow handles long-running human approval and integration work that should remain visible in run history.

Stage	Implementation
Trigger	Dataverse row added or modified for Service Request where approval is required, approval is pending, and integration is unsynced.
Try scope	Start and wait for approval assigned to manager@hellosmart.ca, branch on decision, call mock ERP API, update request, and write External Sync Log.
HTTP sync	POST approved request details to https://api.restful-api.dev/objects and store the returned external id in Dataverse.
Reject branch	Mark approval rejected and skip ERP sync.
Catch scope	Runs after failure, timeout, or skipped Try scope; writes System Error Log with flow run correlation details and marks the request failed.

Resiliency Pattern

- Try/Catch scopes make failures observable and reviewable during the live demo.
- The custom System Error Log captures source component, stage, message, technical detail, correlation ID, and related request.
- Request integration status is updated to failed when the integration path cannot complete.
- Flow run history and Approval records provide proof if tenant email delivery is restricted.

Pro-Code Extensibility

The pro-code pieces are intentionally placed where low-code would either be bypassable or less effective.

Component	Location	Responsibility	Why Here
ServiceRequestRoutingPlugin	src/plugins/ServiceIntake.Plugins/ServiceRequestRoutingPlugin.cs	Evaluates routing rules, assigns department/SLA, sets due date, approval requirement, documentation requirement, and lifecycle defaults.	PreOperation server-side plugin keeps routing transactional and consistent across all entry points.
ServiceRequestClosureGuardPlugin	src/plugins/ServiceIntake.Plugins/ServiceRequestClosureGuardPlugin.cs	Blocks closure of critical requests unless internal resolution notes and documentation evidence are present.	Server-side validation prevents bypass from forms, imports, flows, and APIs.
SlaStatusIndicator PCF	src/pcf/SlaStatusIndicator/	Displays severity, SLA, approval, and sync status as a compact coordinator-facing visual.	PCF improves internal model-driven usability without duplicating authoritative business logic.
Provisioning Utility	src/scripts/ServiceIntake.Provisioning/	Creates metadata, registers plugins, provisions app/forms/views, sample data, and flow definition patching.	Makes the environment build repeatable and keeps ALM steps reviewable.

ALM, Verification, And Demo Evidence

The repository contains both deployable solution packages and unpacked source so reviewers can inspect the implementation.

Deliverables

Artifact	Location
Managed solution	solution/export/Enterprise_ServiceIntake_ForrestZhang_managed.zip
Unmanaged solution	solution/export/Enterprise_ServiceIntake_ForrestZhang_unmanaged.zip
Unpacked solution source	solution/unpacked/managed/ and solution/unpacked/unmanaged/
C# plugin source	src/plugins/ServiceIntake.Plugins/
PCF source	src/pcf/SlasStatusIndicator/
Power Pages source	src/powerpages/ and powerpages-live/
Demo script	docs/demo/demo-script.md
Verification notes	docs/evidence/verification.md

Verified Behavior

Check	Evidence
Plugin build	Passed.
Provisioning utility	Builds and runs, with upstream SDK NU1903 warnings from Microsoft.PowerPlatform.Dataaverse.Client dependency.
PCF	Build and pac pcf push passed.
Power Pages	Dynamic preview shows Finance, 4 hour SLA, approval required, and documentation required for critical funding requests.
Portal submission	Portal demo request created with formatted confirmation SR-20260521-001004.
Closure guard	Smoke test blocked undocumented critical closure and allowed documented closure.
Flow	Active, solution-aware, and includes approval, HTTP sync, reject branch, External Sync Log, and Catch error-log scope.
Solution export/unpack	Managed and unmanaged exports and unpacked source generated successfully.

Known Demo Notes

- If email delivery is restricted, use Approval records and Flow run history as evidence.
- reqres.in currently requires an API key for POST, so the mock ERP endpoint uses api.restful-api.dev to keep the demo self-contained.
- If a target environment does not automatically bind the PCF control on import, add the PCF control to the coordinator form field in the form designer.

Live Demonstration Path

The demo is designed to show the full lifecycle and then drill into implementation decisions.

- 1 Walk through the ERD and architecture choices.
- 2 Submit a critical funding request from the portal and show dynamic preview before submit.

- 3 Open the request in the model-driven app and review routing, SLA, approval, sync status, and internal fields.
- 4 Open the cloud flow and explain approval, HTTP POST, writeback, sync log, reject branch, and Catch scope.
- 5 Demonstrate or explain failure handling through System Error Log rows.
- 6 Demonstrate the critical close guardrail through the smoke test or form behavior.
- 7 Show managed solution, unpacked source, PCF, plugin source, Power Pages source, and GitHub repository.

Prepared Q&A

Question	Answer
Why plugin over flow for routing?	Routing must be transactional and consistent for every entry point, including portal, model app, imports, flows, and APIs.
Why plugin for close validation?	The requirement says agents cannot bypass documentation requirements; server-side PreOperation validation is the correct enforcement point.
Why flow for approvals and ERP sync?	Approvals and integration orchestration need human workflow, connector history, retry visibility, and run evidence.
How do customers only see their data?	Power Pages users are tied to Contacts, and contact-scoped table permissions restrict request visibility to owned rows.
How is this deployable?	Components are solution-aware, exported as managed/unmanaged zips, unpacked with PAC, and source-controlled with raw plugin and PCF code.